



TRABAJO PRÁCTICO

DETECCIÓN DE PHISHING MEDIANTE ANÁLISIS DE URL

RED NEURONAL IMPLEMENTADA CON NUMPY Y SCIKIT-LEARN

ALUMNA: Paz Ariana

MATERIA: Matemática III

CARRERA: Tecnicatura Universitaria en
Programación Informática.

<https://www.kaggle.com/datasets/sahandnamvar/phishing-url-detection-111k-urls-22-features>

(a) Descripción de las columnas

La base de datos es un dataset de detección de phishing basado en características de URLs. Contiene 116.600 registros y 26 columnas detalladas a continuación:

Columna	Texto	Descripción
url	Texto	URL completa.
url_len	Numérica	Longitud total de la URL en caracteres
dom	Texto	Nombre del dominio principal.
dom_len	Numérica	Longitud del nombre de dominio.
is_ip	Binaria	1 si la URL usa IP en lugar de nombre de dominio.
tld	Categoría	Extensión del dominio.
tld_len	Numérica	Longitud de la extensión del dominio.
subdom_cnt	Numérica	Cantidad de subdominios.
letter_cnt	Numérica	Cantidad de letras en la URL.
digit_cnt	Numérica	Cantidad de dígitos numéricos en la URL.
special_cnt	Numérica	Cantidad de caracteres especiales.
eq_cnt	Numérica	Cantidad de signos igual (=).
qm_cnt	Numérica	Cantidad de signos de interrogación (?).
amp_cnt	Numérica	Cantidad de ampersands (&).
dot_cnt	Numérica	Cantidad de puntos (.).
dash_cnt	Numérica	Cantidad de guiones (-).
under_cnt	Numérica	Cantidad de guiones bajos (_).
letter_ratio	Numérica	Proporción de letras sobre el total de caracteres
digit_ratio	Numérica	Proporción de dígitos sobre el total de caracteres.
spec_ratio	Numérica	Proporción de caracteres especiales sobre el total.
is_https	Binaria	1 si la URL usa el protocolo de cifrado
slash_cnt	Numérica	Cantidad de barras (/).
entropy	Numérica	Entropía de Shannon ⁽¹⁾
path_len	Numérica	Longitud del path de la URL.
query_len	Numérica	Longitud de la query string.
label	Binaria	Variable objetivo: 0 = URL legítima, 1 = URL de Phishing.

Distribución de la variable objetivo (label):

Legítimas (0): 100,000 (85.8%)

Phishing (1): 16,600 (14.2%)

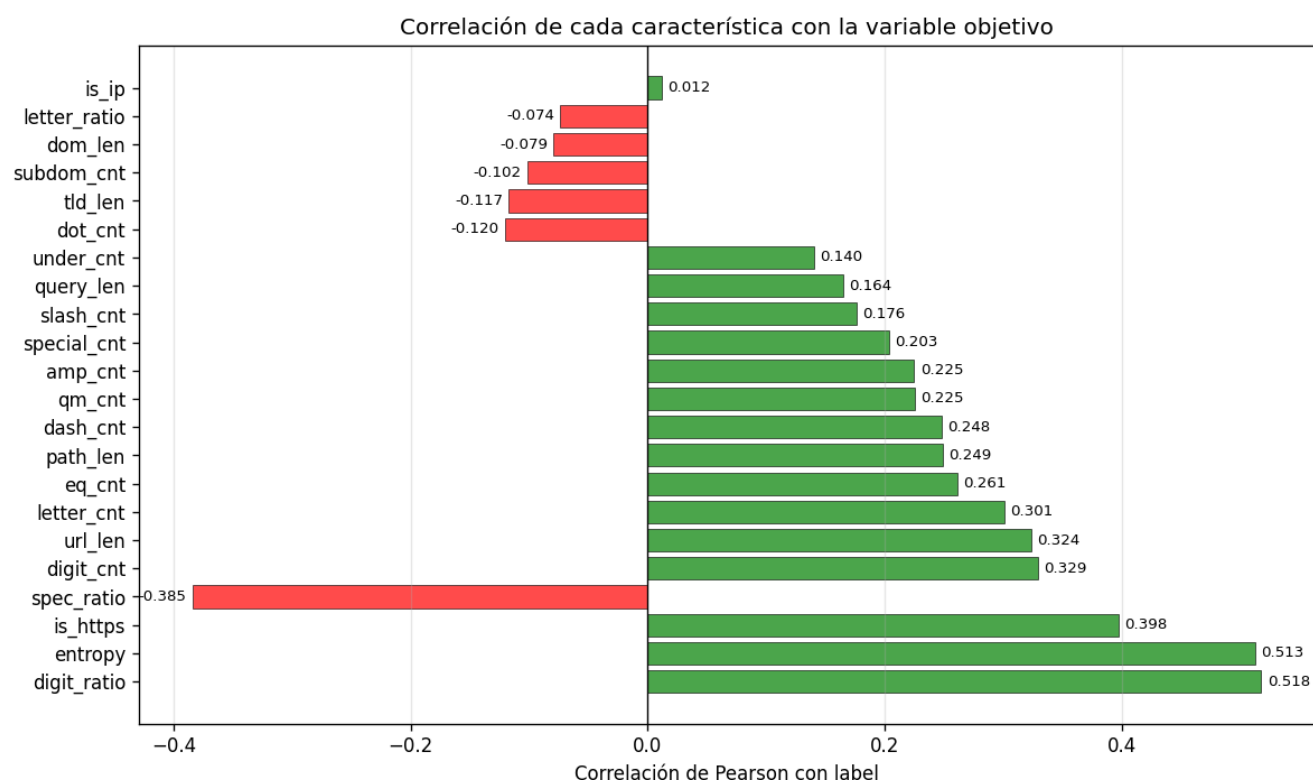
Total : 116,600

¹⁾ Entropía de Shannon: Medición de la aleatoriedad de la url. Cuanto más alto el valor, más alto es el factor de aleatoriedad. Se utiliza para detectar nombres de dominio randomizados ya que algunos sitios utilizan algoritmos que modifican los dominios frecuentemente. Fuente:

<https://www.mdpi.com/1099-4300/23/2/182>

(b) Correlación de características con la variable objetivo

Se calculó la correlación de Pearson entre cada característica numérica y la variable objetivo label. Una correlación positiva alta indica que esa característica se asocian con phishing (label=1).



Conclusión.

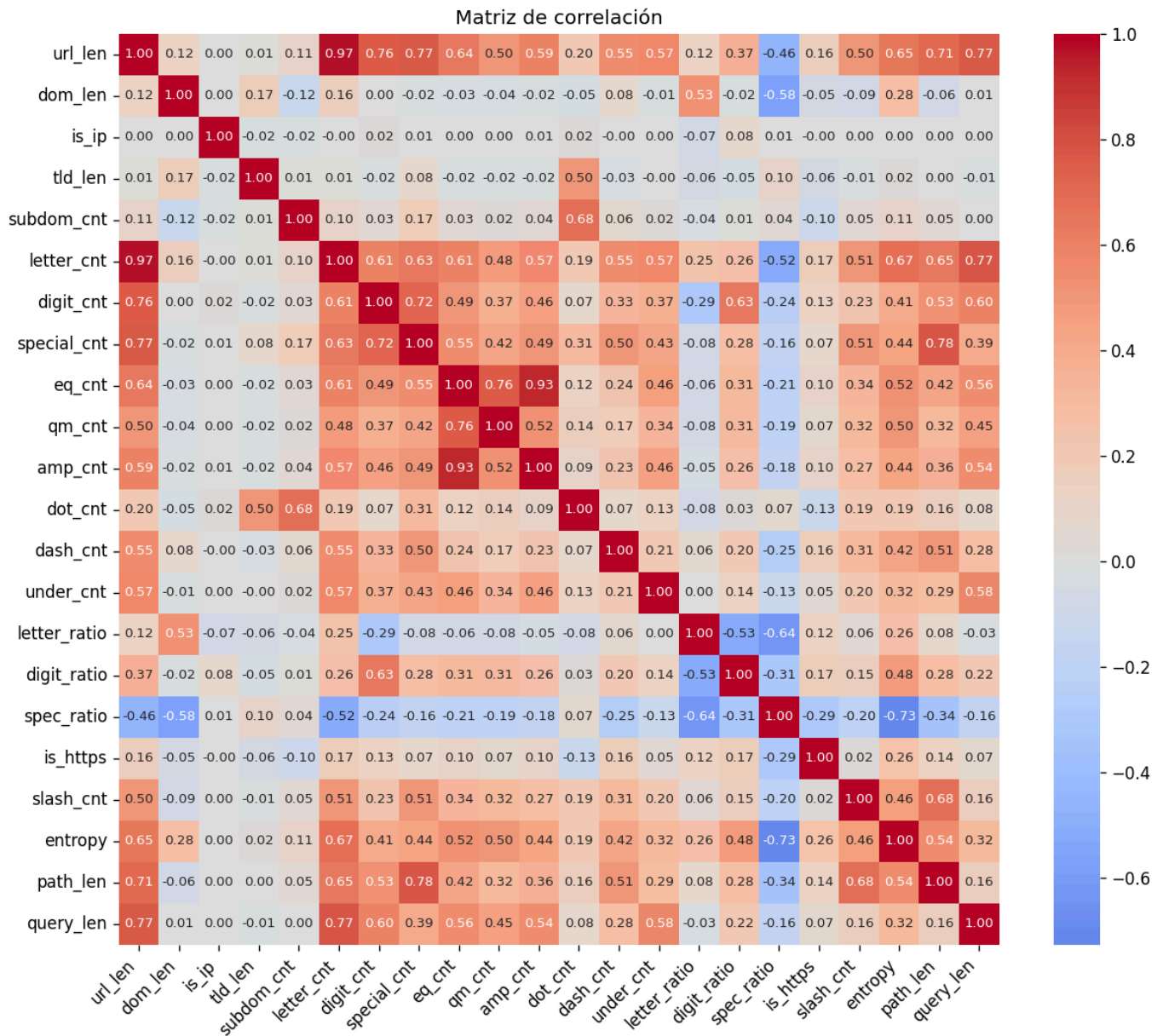
Las características más influyentes son digit_ratio y entropy ya que los atacantes generan subdominios y rutas con cadenas alfanuméricas aleatorias (alta entropía y proporción de dígitos) para evitar blacklist. En base a los resultados, variables relacionadas con la longitud de la URL (url_len, digit_cnt) presentan correlaciones positivas medias. Variables como is_ip o dom_len muestran escasa relación, por lo que su contribución sería limitada. De igual manera, se utilizarán para el entrenamiento. Si causan overfitting, serán re-evaluadas más adelante.

Sorpresivamente, is_https tiene correlación positiva. Esto ocurre porque los atacantes adoptan certificados de seguridad para que las URLs parezcan seguras.⁽²⁾

⁽²⁾Fuente:

<https://www.keyfactor.com/blog/https-phishing-attacks-how-hackers-use-ssl-certificates-to-feign-trust/>

Matriz de correlación entre todas las características.



Conclusión:

Se observan correlaciones positivas altas entre variables de conteo y longitud. Variables como is_ip e is_https muestran baja correlación con el resto, lo que sugiere independencia.

(c) ¿Es esta base de datos adecuada para una red neuronal de clasificación binaria? ¿Qué intentará predecir? ¿Cuál es el objetivo del modelo?

Es una base adecuada para un problema de clasificación binaria por las siguientes razones:

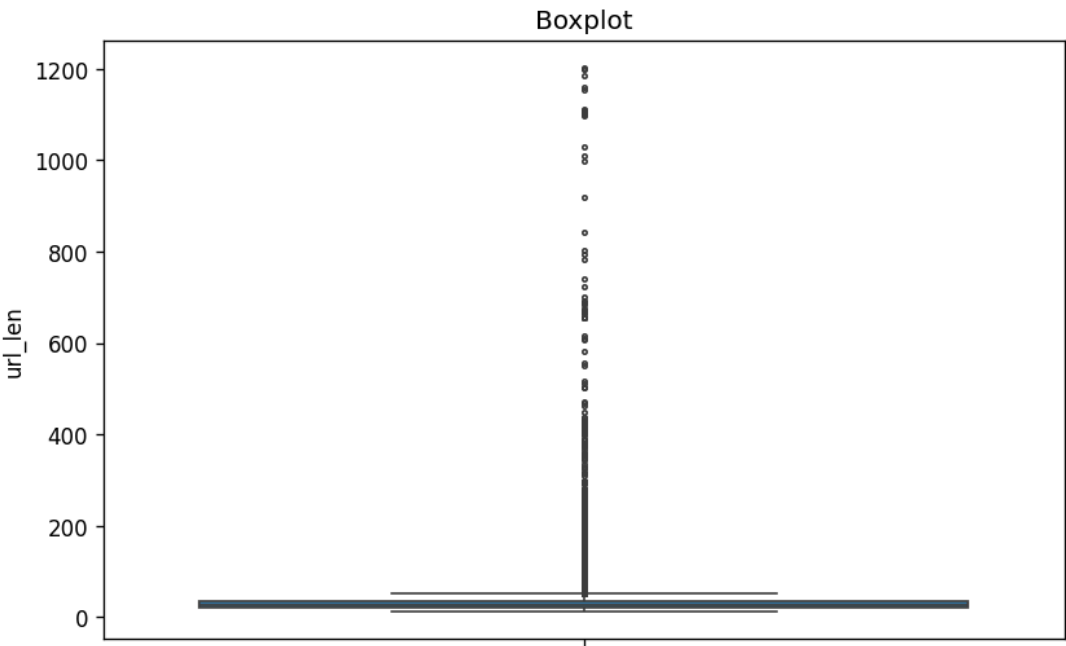
- Variable objetivo binaria: Label puede ser 1 o 0.
- Tamaño: 116.600 registros cómodamente divisible en 70/30 para testing.
- Características: 22 características para alimentar la red.
- Datos reales: Las URLs provienen de fuentes reales (no sintéticas).

Objetivo del modelo: Predecir si una URL es de Phishing (1) o legítima (0) con características extraídas de las URL sin tener que acceder al contenido de la página.

(d) Identificación y limpieza de datos atípicos.

IQR - Sin limpieza.

	Q1	Q3	IQR	Límites	Outliers
url_len	22	34	12	[4.00, 52.00]	9203 (7.89%)
dom_len	9	16	7	[-1.50, 26.50]	1994 (1.71%)
tld_len	2	3	1	[0.50, 4.50]	23655 (20.29%)
subdom_cnt	1	1	0	[1.00, 1.00]	28421 (24.37%)
letter_cnt	16	27	11	[-0.50, 43.50]	7777 (6.67%)
digit_cnt	0	0	0	[0.00, 0.00]	13824 (11.86%)
special_cnt	6	7	1	[4.50, 8.50]	16432 (14.09%)
eq_cnt	0	0	0	[0.00, 0.00]	4581 (3.93%)
qm_cnt	0	0	0	[0.00, 0.00]	4421 (3.79%)
amp_cnt	0	0	0	[0.00, 0.00]	2172 (1.86%)
dot_cnt	2	3	1	[0.50, 4.50]	745 (0.64%)
dash_cnt	0	0	0	[0.00, 0.00]	22877 (19.62%)
under_cnt	0	0	0	[0.00, 0.00]	3212 (2.75%)
slash_cnt	3	3	0	[3.00, 3.00]	34163 (29.3%)
entropy	3.64	4.03	0.38	[3.07, 4.60]	4077 (3.5%)
path_len	1	4	3	[-3.50, 8.50]	20745 (17.79%)
is_ip	0	0	0	[0.00, 0.00]	13 (0.01%)
query_len	0	0	0	[0.00, 0.00]	4417 (3.79%)



Duplicados.

Se encontraron 1.369 filas duplicadas completamente. No aportan información nueva y pueden sesgar el entrenamiento.

Decisión: Se eliminaron las 1.369 filas.

Resultado: 115,231 filas restantes.

Distribución tras limpieza:

Label	Valor	Total
Legítimas	0	98.641
Phishing	1	16.590
Total		115.231

Valores Nulos.

Son 14 filas (<0.01%) solo de la columna tld. Corresponden en su mayoría a URLs con dirección IP (sin extensión de dominio).

Decisión: Se coloca "Unknown" para no perderlas. Esto no afecta la base ya que tld_len queda en 0 y está dentro de los parámetros.

Outliers extremos.

Se encontraron 59 URLs (0.05%) con una longitud mayor a 500 caracteres. Si bien las URLs extensas son señal de Phishing, valores extremos tendrían un impacto negativo.

Decisión: Como forman parte del conjunto de las URLs de Phishing, se acortan al límite superior.

Outliers de conteo.

Se detectaron valores atípicos en columnas como slash_cnt, subdom_cnt y dash_cnt, entre otras. Sin embargo, no se eliminaron porque representan señales genuinas de phishing.

Otras validaciones:

Valores binarios.

Sin valores binarios inválidos,

Columnas de conteo.

No hay valores negativos en la base

Columnas de Ratio.

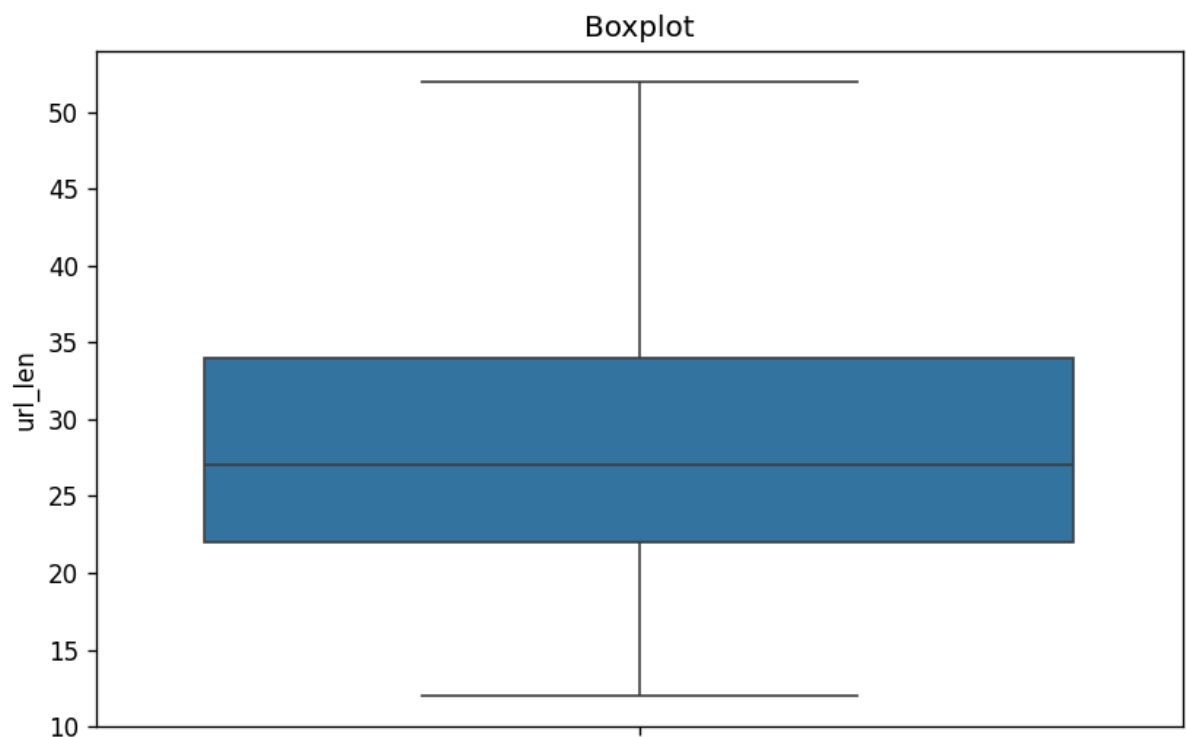
Todas entre 0 y 1.

Consistencia entre datos:

url_len vs len(url), dom_len vs len(dom), tld_len vs len(tld) no presentan inconsistencias.

IQR - Con limpieza.

	Q1	Q3	IQR	Límites	Outliers
url_len	22	34	12	[4.00, 52.00]	0 (0.00%)
dom_len	9	16	7	[-1.50, 26.50]	1984 (1.72%)
tld_len	2	3	1	[0.50, 4.50]	23551 (20.44%)
subdom_cnt	1	1	0	[1.00, 1.00]	28282 (24.54%)
letter_cnt	16	27	11	[-0.50, 43.50]	7747(6.72%)
digit_cnt	0	0	0	[0.00, 0.00]	13775 (11.95%)
special_cnt	6	7	1	[4.50, 8.50]	16340 (14.18%)
eq_cnt	0	0	0	[0.00, 0.00]	4560 (3.96%)
qm_cnt	0	0	0	[0.00, 0.00]	4400 (3.82%)
amp_cnt	0	0	0	[0.00, 0.00]	2171 (1.88%)
dot_cnt	2	3	1	[0.50, 4.50]	742 (0.64%)
dash_cnt	0	0	0	[0.00, 0.00]	22741 (19.74%)
under_cnt	0	0	0	[0.00, 0.00]	3177 (2.76%)
slash_cnt	3	3	0	[3.00, 3.00]	33905 (29.42%)
entropy	3.64	4.03	0.38	[3.07, 4.60]	4067 (3.53%)
path_len	1	4	3	[-3.50, 8.50]	20599 (17.88%)
is_ip	0	0	0	[0.00, 0.00]	13 (0.01%)
query_len	0	0	0	[0.00, 0.00]	4396 (3.81%)



(d) Realizá la normalización de los datos que lo requieran. Justificá el método utilizado.

SPLIT DE DATOS⁽³⁾

Antes de realizar la normalización, se realizó la división de la base.

- 70% para entrenamiento.
- 30% para testing.

Resultados:

Set de training: 80,661 muestras (70%)

Set de test: 34,570 muestras (30%)

Distribución de clases - Train:

Clase 0: 69,048 (85.6%)

Clase 1: 11,613 (14.4%)

Distribución de clases - Test:

Clase 0: 29,593 (85.6%)

Clase 1: 4,977 (14.4%)

Possible Inconveniente futuro: Desbalance de clases.

NORMALIZACIÓN

Se aplicó Min - Max Scaling sobre las características numéricas por las siguientes razones:

1. Compatibilidad. Las funciones de activación Relu/Sigmoide se entrenan mejor con entradas de rango [0, 1]
2. Heterogeneidad: Algunas columnas tienen rangos muy diferentes.

⁽³⁾<https://medium.com/@spinjosovsky/normalize-data-before-or-after-split-of-training-and-testing-data-7b8005f81e26>

Parte 2 — Red Neuronal.

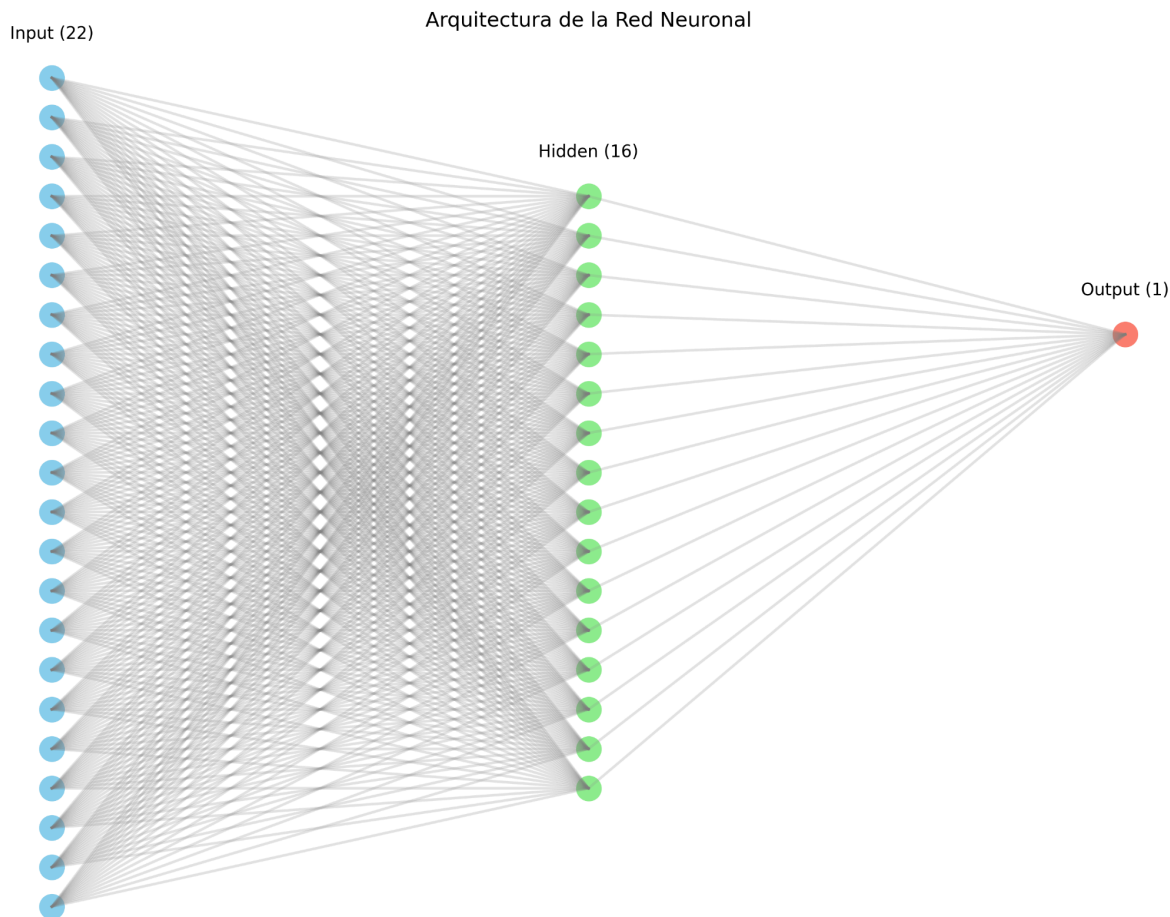
(a) Dibujá la arquitectura de la red. Indicá la cantidad de neuronas de cada capa y las funciones de activación elegidas. Justificar.

Funciones:

- ReLu - capa oculta: Evita el gradiente desvaneciente además de ser eficiente y no lineal.
- Sigmoide en salida: La salida se interpreta como probabilidad ya que produce un valor de 0 o 1. En este caso, $P(\text{phishing}) \geq 0.5 \rightarrow 1$.

Neuronas ocultas: Se eligieron 16 para una mayor representación intentando evitar overfitting.

Fuente: <https://www.datacamp.com/es/tutorial/introduction-to-activation-functions-in-neural-networks>
<https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/>



ARQUITECTURA DE LA RED NEURONAL

Se definió:

- Capa de entrada: 22 neuronas.
- Capa oculta: 16 neuronas — activación: ReLU.
- Capa de salida: 1 neurona — activación: Sigmoide.

Parámetros totales:

- $W1: 22 \times 16 = 352$ pesos + 16 bias.
- $W2: 16 \times 1 = 16$ pesos + 1 bias
- Total: 385 parámetros

(b) Implementa tu red neuronal utilizando solo la librería numpy. No está permitido el uso de librerías de machine learning para esta parte. Asegúrate de incluir: Inicialización de los pesos, cálculo de las activaciones para cada capa, función de costo, algoritmo de retropropagación (backpropagation) y ajuste de los pesos mediante descenso por gradiente estocástico.

Los pesos fueron inicializados mediante He Initialization, técnica adecuada para redes con activación ReLU, ya que mantiene una varianza estable de las activaciones a través de las capas. Los sesgos se inicializan en cero.

Posteriormente se implementó la etapa de forward propagation. En esta fase cada muestra atraviesa la red calculando primero la combinación lineal de la capa oculta:

$$Z1 = XW1 + b1$$

Luego se aplica la función ReLU para obtener las activaciones intermedias:

$$A1 = \text{ReLU}(Z1)$$

Finalmente se calcula la salida de la red:

$$Z2 = A1W2 + b2$$

y se transforma mediante una función sigmoide para obtener una probabilidad de pertenecer a la clase phishing:

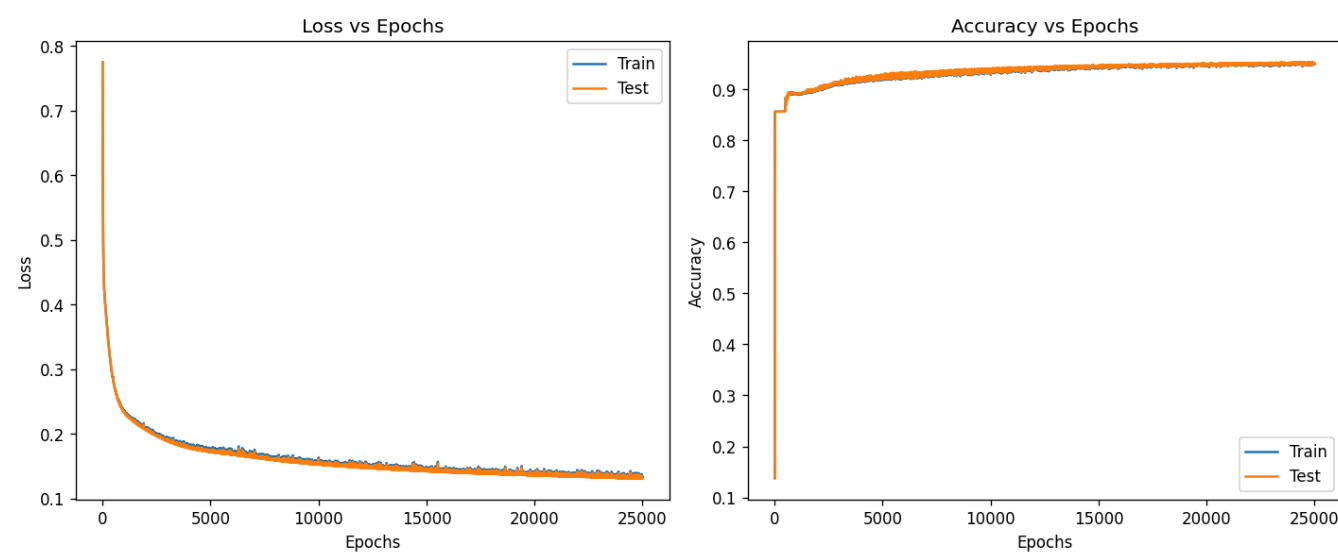
$$A2 = \text{sigmoid}(Z2)$$

El cálculo de gradientes se realizó mediante el algoritmo de retropropagación, aplicando la regla de la cadena para determinar la contribución de cada peso al error final.

Una vez obtenidos los gradientes, los parámetros se actualizaron utilizando Descenso por Gradiente Estocástico (SGD). En cada iteración se seleccionó una muestra aleatoria del conjunto de entrenamiento, se calculó el error, se hizo backward propagation y se ajustaron los pesos y sesgos en dirección opuesta al gradiente para minimizar la función de costo.

(4)https://en.wikipedia.org/wiki/Weight_initialization
<https://machinelearningmastery.com/weight-initialization-for-deep-learning-neural-networks/>
<https://medium.com/@piyushkashyap045/understanding-weight-initialization-techniques-in-neural-networks-582e80a1e839>

(c) Entrena tu red con la base de datos seleccionada. Traza las curvas de precisión (accuracy) y la función de pérdida para el set de entrenamiento y el set de test a lo largo de las iteraciones (epochs).



Loss	
Train	0.1336
Test	0.1314
Accuracy	
Train	0.9499
Test	0.9512

Valores utilizados:

- Tasa de aprendizaje = 0.05
- Epochs = 25000
- Batch Size = 64

Evaluación de los gráficos:

Inicio:

Predice todo 1 $\rightarrow$ 0.14

Primeras épocas:

Aprende que la mayoría es 0.

A medida que avanza.

Empieza a predecir los casos de Phishing.

Resultado:

Accuracy sube a $\sim$ 0.95

Antes del entrenamiento, el modelo tiende a predecir mayormente la clase minoritaria, lo que resulta en una baja precisión inicial. Esto se debe a la inicialización aleatoria de los pesos.

A medida que avanza el entrenamiento, el modelo aprende la distribución de las clases y mejora rápidamente. Este comportamiento también evidencia el desbalance en el dataset, donde la clase 0 representa aproximadamente el 85% de los datos.

Durante las pruebas se observó que entrenar la red neuronal durante aproximadamente 3 épocas resultaba insuficiente. Al incrementar el entrenamiento a aproximadamente 20 épocas, el recall de la clase phishing aumentó de 0.59 a 0.77 y el accuracy global de 0.92 a 0.95.

Corrección del cálculo de gradientes (promedio por batch)⁽⁵⁾

Inicialmente, los gradientes no estaban siendo normalizados por el tamaño del batch, lo que generaba actualizaciones de pesos demasiado grandes e inestables.

Se corrigió dividiendo por el tamaño del batch (m)

Esto asegura que las actualizaciones sean proporcionales al tamaño del lote de datos utilizado, estabilizando el proceso de entrenamiento.

⁽⁵⁾Fuente: <https://www.ultralytics.com/es/glossary/batch-normalization>

(d) Analiza si tu modelo presenta overfitting. ¿Cómo se puede identificar el punto en el que el modelo empieza a sobreajustarse a los datos de entrenamiento? ¿Cómo lo manejarías?

El overfitting puede identificarse cuando la pérdida de entrenamiento continúa disminuyendo mientras que la pérdida de validación comienza a aumentar.

En este caso no se observó dicho comportamiento. Las curvas de entrenamiento y validación evolucionan de manera similar. Además, la pérdida permanece cercana en ambos conjuntos, indicando una buena capacidad de generalización.

Parte 3 - Comparación con scikit-learn

(a) Utiliza scikit-learn para implementar una red neuronal similar a la que desarrollaste manualmente. Asegúrate de mantener la misma cantidad de capas y neuronas, y utilizar las mismas funciones de activación.

Se implementó la misma red neuronal usando MLPClassifier de scikit-learn, manteniendo la misma arquitectura: 1 capa oculta con 16 neuronas, activación ReLU, Descenso de Gradiente Estocástico, learning rate 0.05, 25000 epoch, mini-batches de 64.

Resultado.

Accuracy Train: 0.9710
Accuracy Test: 0.9720

(b) Compara el rendimiento de la red neuronal implementada en numpy con la que implementaste en scikit-learn. ¿En qué aspectos los resultados son similares? ¿En qué aspectos son diferentes?

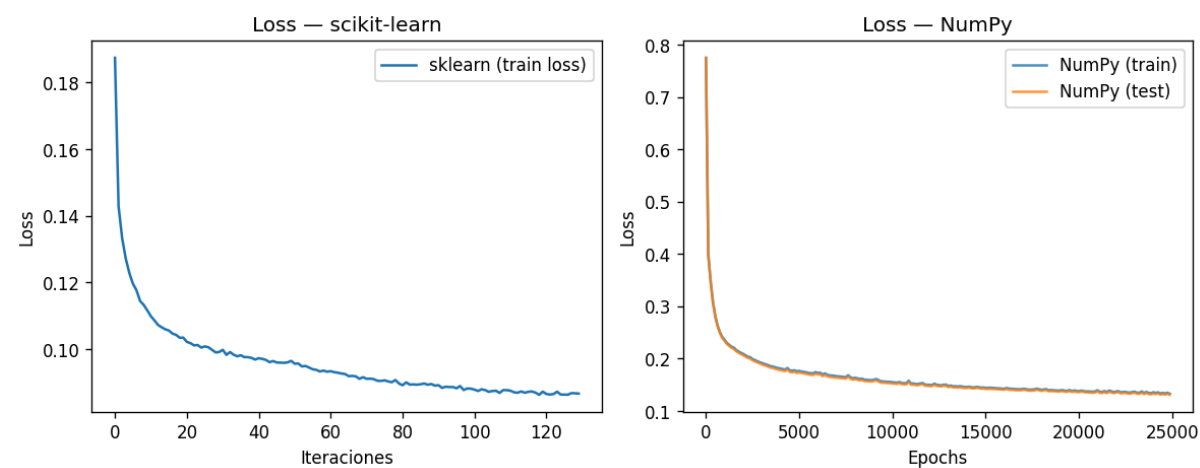
La comparación entre ambas implementaciones muestra resultados similares, aunque la red desarrollada con scikit-learn mantiene una ventaja en todas las métricas evaluadas.

La implementación manual en NumPy alcanzó una precisión global del 95%, mientras que scikit-learn obtuvo aproximadamente un 97%. Para la clase phishing, la red NumPy logró una precisión de 0.87, recall de 0.77 y F1-score de 0.82. Por su parte, scikit-learn obtuvo una precisión de 0.92, recall de 0.88 y F1-score de 0.90.

Inicialmente las diferencias eran considerablemente mayores al utilizar menos epochs. Sin embargo, al aumentarlas, la implementación en NumPy mejoró significativamente, especialmente en la detección de URLs de phishing.

Las diferencias restantes pueden atribuirse a optimizaciones internas implementadas por scikit-learn.

Accuracy	NumPy	Sklearn
Train	0.9497	0.9710
Test	0.9510	0.9720



	precision	recall	f1 Score	support
0 - Numpy	0.96	0.98	0.97	29593
0 - Scikit	0.98	0.99	0.98	
1 - Numpy	0.87	0.77	0.82	4977
1 - Scikit	0.92	0.88	0.90	

(c) En esta parte puedes analizar otras arquitecturas modificando parámetros como cantidad de capas ocultas, cantidad de neuronas, etc. Justificar el análisis de rendimiento

Arquitectura	Accuracy Test
A1 - Original (16,)	0.9719
A2 - Dos Capas (32, 16)	0.9757
A3 - Tres Capas (64, 32, 16)	0.9762

Se observa una mejora progresiva. Pasar de una capa oculta a dos capas implicó una ganancia de ~0.38%, mientras que agregar una tercera capa sumó apenas ~0.05% adicional. A mayor cantidad de parámetros, también aumenta el riesgo de overfitting y el tiempo de entrenamiento, por lo que la arquitectura A1 representa un buen balance entre simplicidad y rendimiento. Cabe destacar que el aumento de complejidad no elimina el desbalance de clases.

Parte 3 - Conclusión.

Respuesta: Es posible detectar phishing analizando la URL.

El dataset presenta un desbalance notable: 85.6% de URLs legítimas vs 14.4% de phishing. Si bien el modelo alcanza un accuracy del ~95%, esta métrica puede ser engañosa en datasets desbalanceados, ya que un clasificador que predijera siempre clase 0 obtendría un 85.6% de accuracy.

Impacto del análisis.

- Limpieza: URLs duplicadas, valores nulos en tld y URLs largas permitió entrenar datos representativos.
- Evaluación de correlación. Esto reveló que digit_ratio y entropy son las más representativas.
- Normalización. Permitted gradientes equilibrados y mejor uso de activación.

Conocimientos consolidados.

- Forward pass. Envío de información entre capas con matrices.
- Funciones de activación. Por qué Relu en Capa oculta y sigmoide en la salida.
- Binary Cross Entropy. Para bases de clasificación binaria.
- Forward y Backpropagation. Cómo calcularlas y qué función cumple cada una.

ReLU en capa oculta: evita el gradiente desvaneciente.

Sigmoide en salida: La salida se interpreta como probabilidad ya que produce un valor de 0 o 1. En este caso, $P(\text{phishing}) \geq 0.5 \rightarrow 1$.

Neuronas ocultas: 16 para una mayor representación intentando evitar overfitting.

Aclaración:

La IA contribuyó a la mejor visualización de los gráficos en base al código básico proporcionado. Ejemplo: El primer gráfico de correlación o de la red neuronal.

Fuentes Adicionales:

<https://wandb.ai/ayush-thakur/dl-question-bank/reports/ReLU-vs-Sigmoid-Function-in-Deep-Neural-Networks--VmlldzoyMDk0Mzl>

Archivos .ipynb vistos en clase.